

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_diabetes
```

Load Dataset

```
In [ ]: data = load_diabetes()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
```

Understand the structure

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 442 entries, 0 to 441
Data columns (total 11 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0    age    442 non-null    float64
 1    sex     442 non-null    float64
 2    bmi     442 non-null    float64
 3    bp      442 non-null    float64
 4    s1      442 non-null    float64
 5    s2      442 non-null    float64
 6    s3      442 non-null    float64
 7    s4      442 non-null    float64
 8    s5      442 non-null    float64
 9    s6      442 non-null    float64
10   target  442 non-null    float64
dtypes: float64(11)
memory usage: 38.1 KB
```

```
In [ ]: df.head()
```

```
Out[ ]:      age    sex    bmi    bp    s1    s2    s3    s4
0  0.038076  0.050680  0.061696  0.021872 -0.044223 -0.034821 -0.043401 -0.002592
1 -0.001882 -0.044642 -0.051474 -0.026328 -0.008449 -0.019163  0.074412 -0.039493
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -0.032356 -0.002592
3 -0.089063 -0.044642 -0.011595 -0.036656  0.012191  0.024991 -0.036038  0.034309
4  0.005383 -0.044642 -0.036385  0.021872  0.003935  0.015596  0.008142 -0.002592
```



Handle Missing Values

```
In [ ]: df.isnull().sum()
```

```
Out[ ]:      0
      age  0
      sex  0
      bmi  0
      bp   0
      s1   0
      s2   0
      s3   0
      s4   0
      s5   0
      s6   0
      target  0
```

dtype: int64

Remove Duplicates

```
In [ ]: df.duplicated().sum()
```

```
Out[ ]: np.int64(0)
```

```
In [ ]: df = df.drop_duplicates()
```

Summary Statistics

```
In [ ]: df.describe()
```

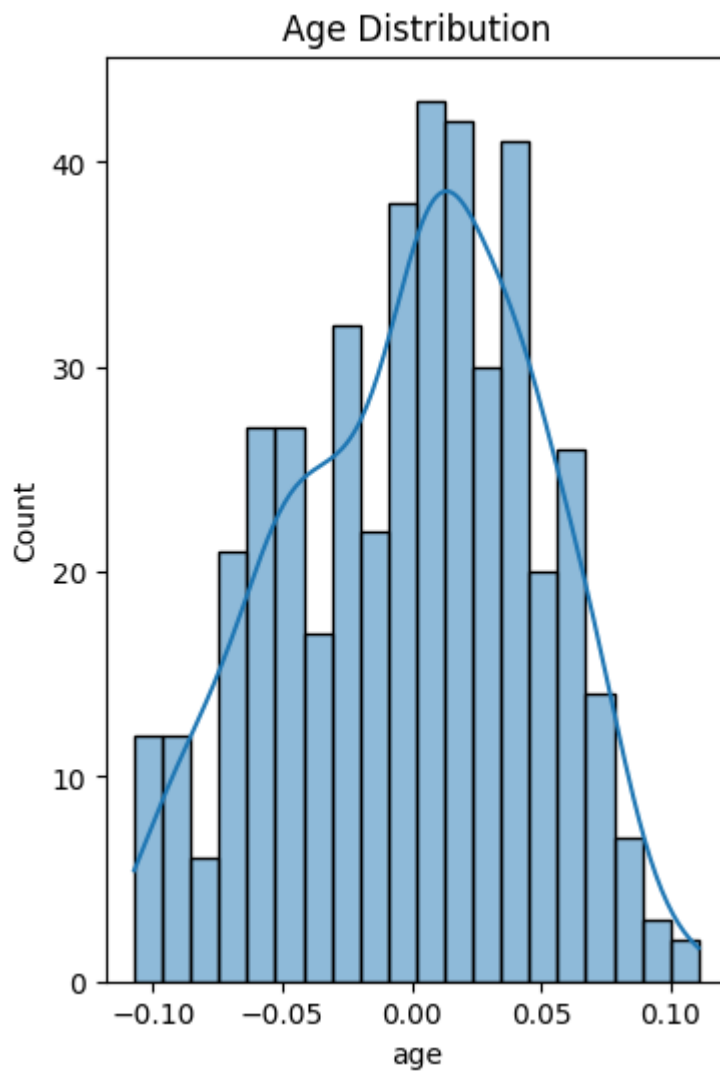
Out[]:

	age	sex	bmi	bp	s1	
count	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02
mean	-2.511817e-19	1.230790e-17	-2.245564e-16	-4.797570e-17	-1.381499e-17	3.91843
std	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.76190
min	-1.072256e-01	-4.464164e-02	-9.027530e-02	-1.123988e-01	-1.267807e-01	-1.156
25%	-3.729927e-02	-4.464164e-02	-3.422907e-02	-3.665608e-02	-3.424784e-02	-3.035
50%	5.383060e-03	-4.464164e-02	-7.283766e-03	-5.670422e-03	-4.320866e-03	-3.819
75%	3.807591e-02	5.068012e-02	3.124802e-02	3.564379e-02	2.835801e-02	2.98443
max	1.107267e-01	5.068012e-02	1.705552e-01	1.320436e-01	1.539137e-01	1.98788

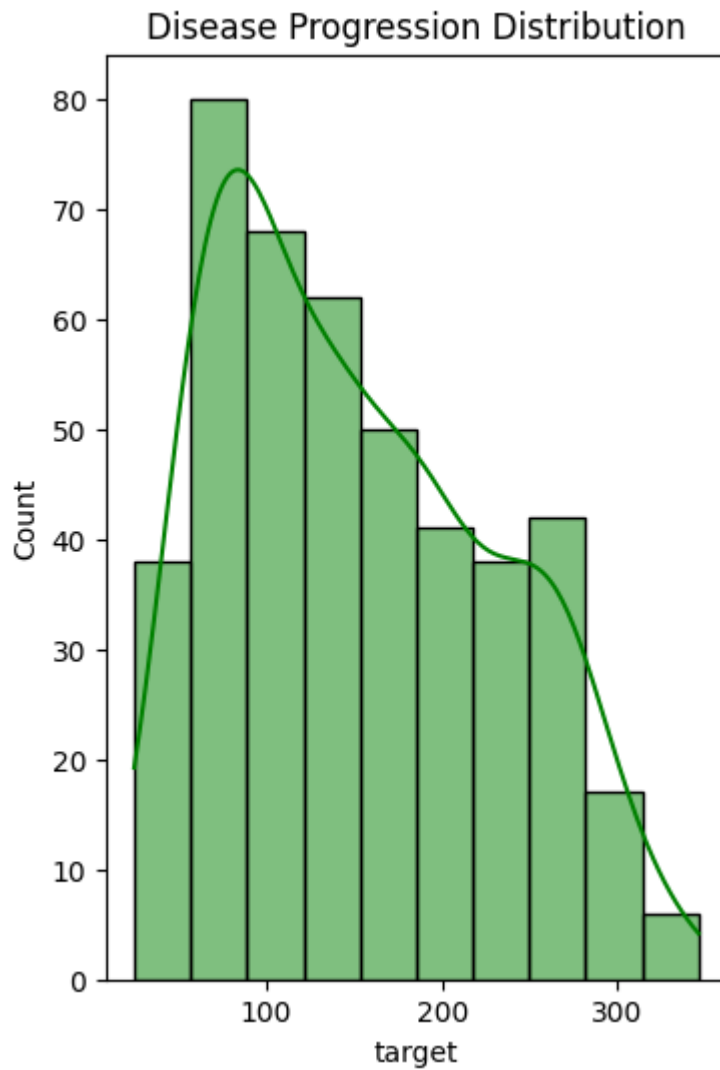


Univariate Analysis

```
In [ ]: ### Histogram for age
plt.figure(figsize=(4,6))
sns.histplot(df['age'], kde=True, bins=20)
plt.title('Age Distribution')
plt.show()
```

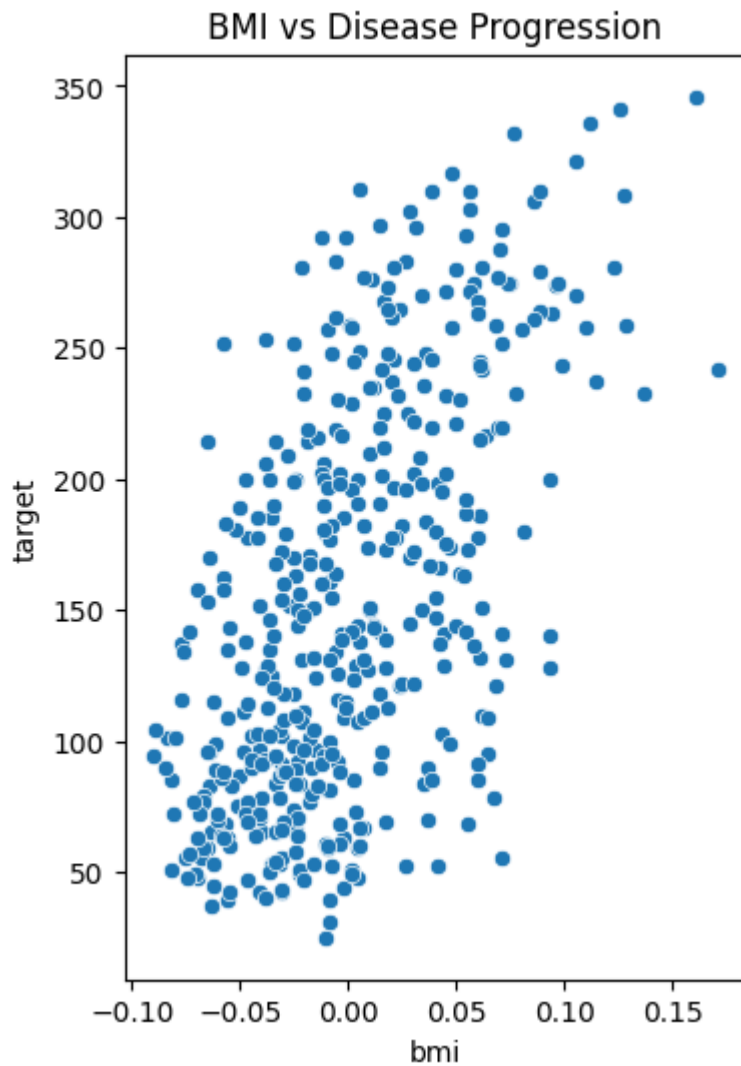


```
In [ ]: ### Histogram for target = disease propagation
plt.figure(figsize=(4,6))
sns.histplot(df['target'], kde=True, color="green")
plt.title('Disease Progression Distribution')
plt.show()
```

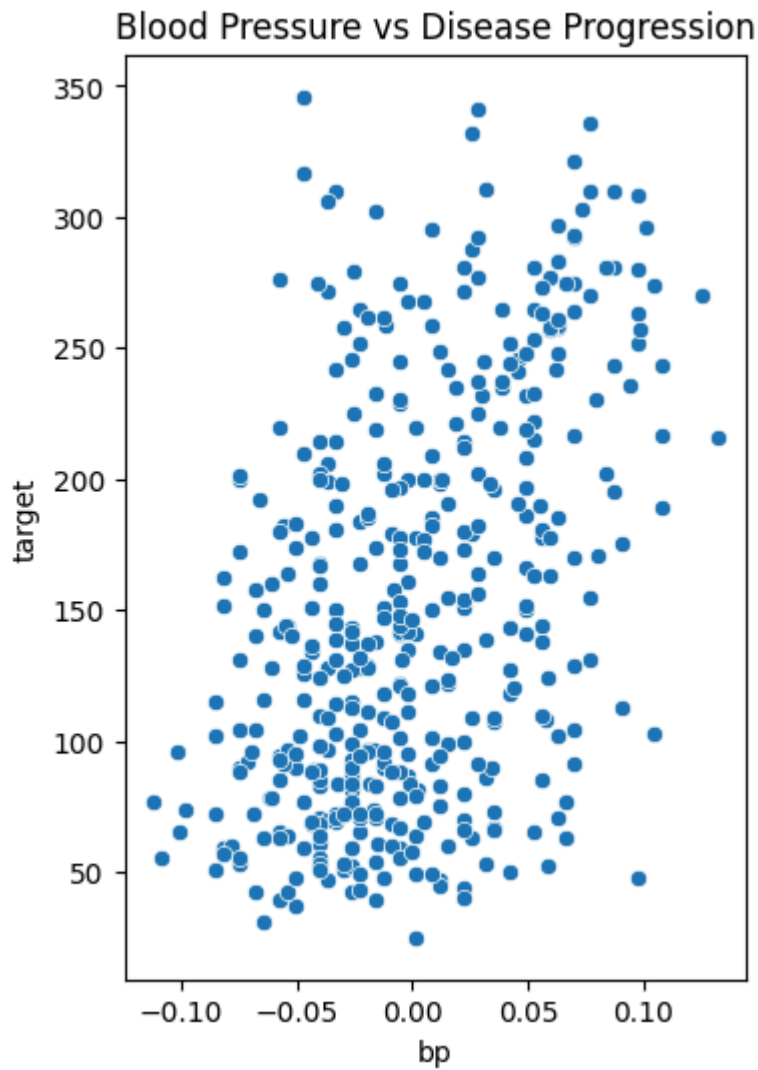


Bivariate Analysis

```
In [ ]: ### BMI vs Target  
plt.figure(figsize=(4,6))  
sns.scatterplot(x='bmi', y='target', data=df)  
plt.title('BMI vs Disease Progression')  
plt.show()
```

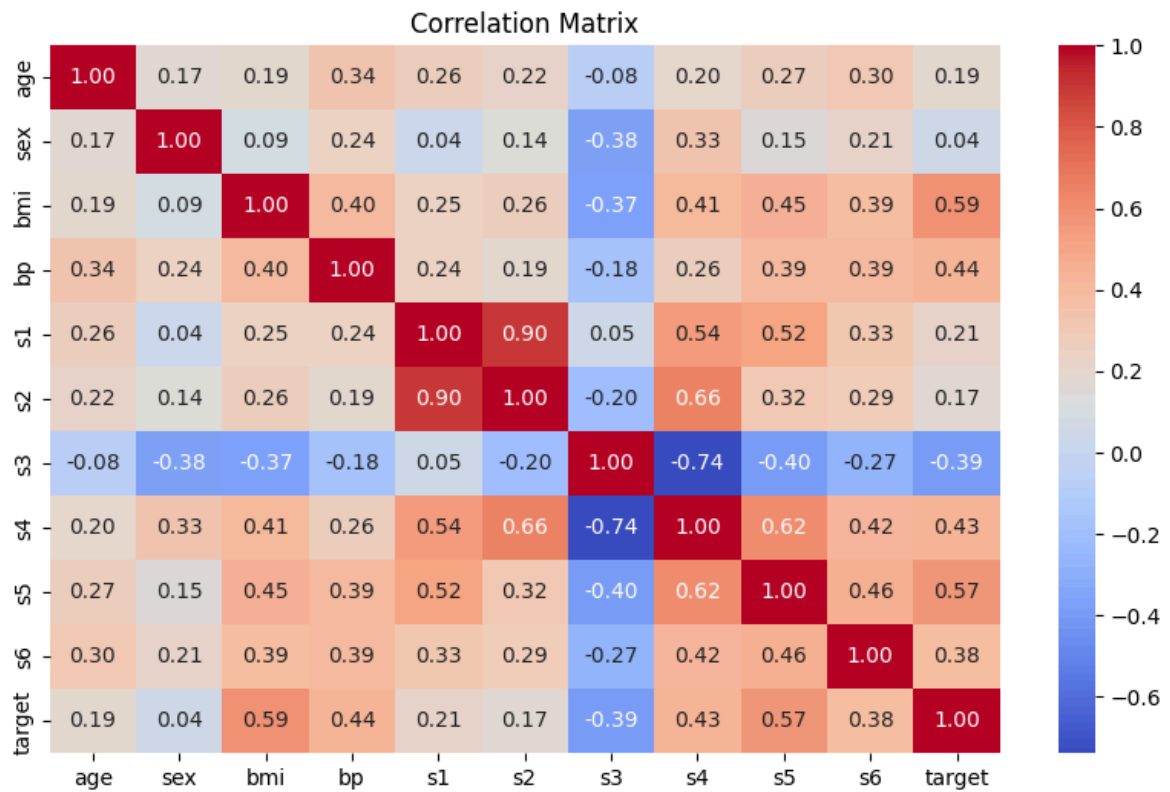


```
In [ ]: ### Blood Pressure vs Target
plt.figure(figsize=(4,6))
sns.scatterplot(x='bp', y='target', data=df)
plt.title('Blood Pressure vs Disease Progression')
plt.show()
```



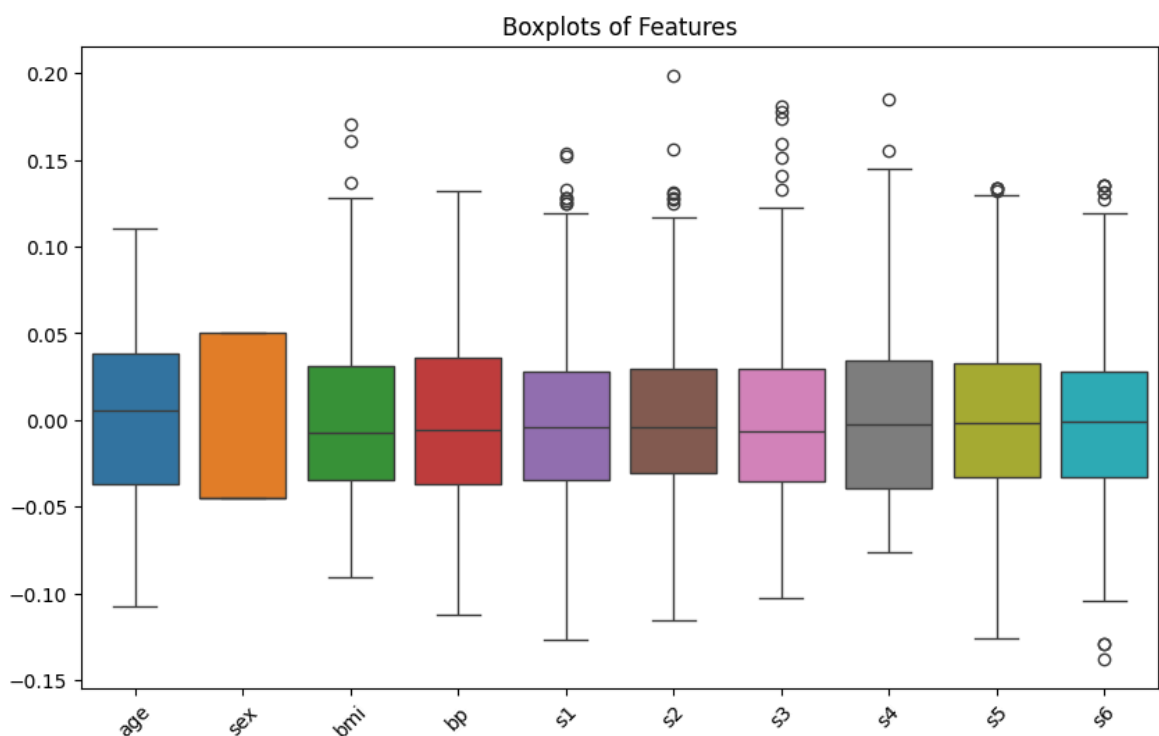
Multivariate Analysis

```
In [ ]: ### Correlation Matrix
plt.figure(figsize=(10,6))
corr = df.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix')
plt.show()
```



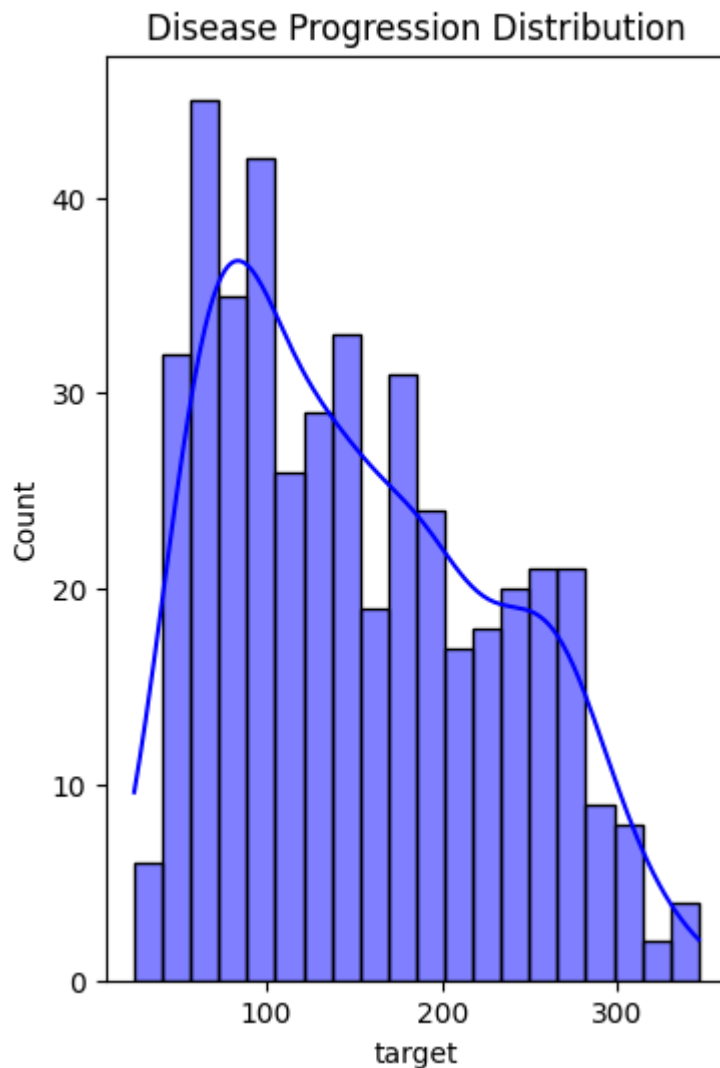
Outlier Detection

```
In [ ]: ### Boxplots
plt.figure(figsize=(10,6))
sns.boxplot(df.drop('target', axis=1))
plt.title('Boxplots of Features')
plt.xticks(rotation=45)
plt.show()
```



Analyze target variable


```
In [ ]: plt.figure(figsize=(4,6))
sns.histplot(df['target'], kde=True, color="blue", bins=20)
plt.title('Disease Progression Distribution')
plt.show()
```



Conclusion

In this lab, we performed EDA on healthcare datasets to uncover patterns, trends, and anomalies. Through visualizations and statistics, we gained critical insights into patient health data, laying a strong foundation for informed machine learning model development.

Postlab

1. What is Exploratory Data Analysis (EDA)?

EDA (Exploratory Data Analysis) is the process of visually and statistically analyzing a dataset to:

- Understand key characteristics
- Detect **patterns**, **anomalies**, and **outliers**

- Gain insights before building ML models
- Prepare the data for further processing

2. Why is EDA important in healthcare data analysis?

EDA helps in:

- Identifying **trends** (e.g., disease risk by age or gender)
- Detecting **missing values** or **errors** in clinical records
- Understanding **relationships** (e.g., BMI vs Glucose levels)
- Revealing **class imbalance** in disease datasets
- Ensuring that model inputs are **valid**, **clean**, and **relevant**

3. Common Python Libraries Used for EDA in Healthcare:

Library	Use Case
pandas	Data manipulation and summaries
matplotlib	Basic visualizations (line, bar)
seaborn	Advanced statistical plots
numpy	Numerical operations
scikit-learn	Dataset loading, correlation checks, preprocessing

4. Challenges in Performing EDA on Healthcare Data:

- **Missing or Incomplete Data**
- **Inconsistent Units** (e.g., mg/dL vs mmol/L for glucose)
- **Data Privacy & Anonymization** concerns
- **High Dimensionality** (many lab tests, vitals, scans)
- **Unstructured Medical Text** (doctor’s notes, discharge summaries)